

Experimental evaluation shows that WARP achieves a 41x reduction in end-to-end latency compared to the XTR reference implementation on LoTTE Pooled, bringing query response times down from above 6 seconds to just 171 milliseconds in single-threaded execution, while also reducing index size by a factor of 2x–4x compared to the ScaNN-based baseline. Furthermore, WARP demonstrates a 3x speedup over the state-of-the-art ColBERTv2/PLAID system, as illustrated in Figure 1.

After reviewing prior work on efficient neural IR in Section 2, we analyze the latency bottlenecks in the ColBERT and XTR retrieval frameworks in Section 3, identifying key areas for optimization. These findings form the foundation for our work on WARP, which we introduce and describe in detail in Section 4. In Section 5, we evaluate WARP’s end-to-end latency and scalability using the BEIR [20] and LoTTE [19] benchmarks. Finally, we compare our implementation to existing state-of-the-art engines.

## 2 Related Work

Dense retrieval models can be broadly categorized into single-vector and multi-vector approaches. Single-vector methods, exemplified by ANCE [21] and STAR/ADORE [22], encode a passage into a single dense vector [7]. While these techniques offer computational efficiency, their inherent limitation of representing complex documents with a single vector has been shown to constrain the model’s ability to capture intricate information structures [8].

To address such limitations, ColBERT [8] introduces a multi-vector paradigm. Here, both queries and documents are independently encoded as multiple embeddings, allowing for a richer representation of document content and query intent. The multi-vector approach is further refined in ColBERTv2 [19], which improves supervision and incorporates residual compression to reduce the space requirements associated with storing multiple vectors per indexed document. Building upon these innovations, PLAID [18] substantially accelerates ColBERTv2 by efficiently pruning non-relevant passages using the residual representation and by employing optimized C++ kernels for decompression and scoring. EMVB [15] further optimizes PLAID’s memory usage and *single-threaded* query latency using product quantization [6] and SIMD instructions.

Separately, COIL [2] incorporates insights from conventional retrieval systems [17] by constraining token interactions to lexical matches between queries and documents. SPLATE [1] translates the embeddings produced by ColBERTv2 style pipelines to a sparse vocabulary, allowing the candidate generation step to be performed using traditional *sparse* retrieval techniques. CITADEL [13] introduces conditional token interaction through dynamic lexical routing, selectively considering tokens for relevance estimation. While CITADEL significantly reduces GPU execution time, it falls short of PLAID’s CPU performance at comparable retrieval quality.

The ConteXtualized Token Retriever (XTR) [11] represents a notable conceptual advancement in dense retrieval. XTR simplifies the scoring process and eliminates the gathering stage entirely, *theoretically* enhancing retrieval efficiency. However, its current end-to-end latency limits its applicability in production environments, where even minor increases in query response time can degrade user experience and negatively affect revenue [9].

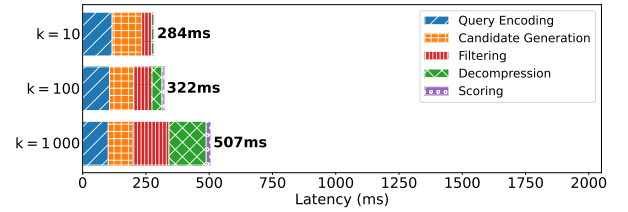


Figure 2: Breakdown of ColBERTv2/PLAID’s avg. latency for varying  $k$  on LoTTE Pooled (Dev Set)

## 3 Latency of Current Neural Retrievers

We start by analyzing two state-of-the-art multi-vector retrieval methods to identify their bottlenecks, providing the foundation for our work on WARP. We evaluate the latency of PLAID and XTR across various configurations and datasets: BEIR NFCorpus, LoTTE Lifestyle, and LoTTE Pooled. In XTR, token retrieval emerges as a fundamental bottleneck: the need to retrieve a large number of candidates from the ANN backend significantly impacts performance. PLAID, while generally more efficient, faces challenges in its decompression stage. Query encoding emerges as a shared limitation for both engines, particularly on smaller datasets. These insights inform the design of WARP, which we introduce in Section 4.

### 3.1 ColBERTv2/PLAID

As shown in Figure 2, we evaluate PLAID’s performance using its optimized implementation [4] and the ColBERTv2 checkpoint from Hugging Face [3]. We configure PLAID’s hyperparameters similar to the original paper [18]. Consistent with prior work [15], we observe single threaded CPU latency exceeding 500ms on LoTTE Pooled. Furthermore, we find that the decompression stage remains rather constant for fixed  $k$  across all datasets, consuming approximately 150–200ms for  $k = 1000$ . Notably, for smaller datasets like BEIR NFCorpus and large  $k$  values, this stage constitutes a significant portion of the overall query latency. Thus, the decompression stage emerges as a critical bottleneck for small datasets. As anticipated, the filtering stage’s execution time is proportional to the number of candidates, increasing for larger  $k$  values and bigger datasets. In contrast, candidate generation constitutes a fixed cost based on the number of centroids. Interestingly, the scoring stage appears to have a negligible impact on ColBERTv2/PLAID’s overall latency across all measurements.

### 3.2 XTR/ScaNN

To enable benchmarking of the XTR framework, we develop a Python library based on Google DeepMind’s published code [12]. We provide the library’s code and scripts to reproduce the benchmarks on GitHub<sup>2</sup>. Unless otherwise specified, all benchmarks utilize the XTR BASE\_EN transformer model for encoding. This model was published and is available on Hugging Face [10]. In accordance with the paper [11], we evaluate the implementation for  $k' = 1000$  and  $k' = 40000$ .

<sup>2</sup><https://github.com/jlscheerer/xtr-eval>

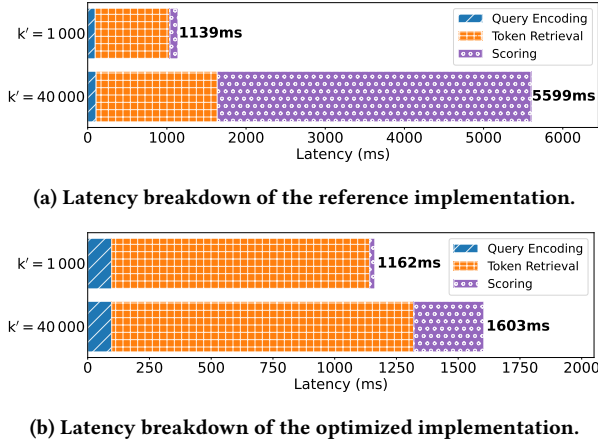


Figure 3: Breakdown of XTR<sub>base</sub>/ScaNN on LoTTE Pooled.

As seen in Figure 3a, the scoring stage constitutes a significant bottleneck in the end-to-end latency of the XTR framework, particularly when dealing with large values of  $k'$ . We argue that this performance bottleneck is largely attributed to an *unoptimized* implementation in the released code, which relies on native Python data structures and manual iteration, introducing substantial overhead, especially for large numbers of token embeddings. We refactor this implementation to leverage optimized data structures and vectorized operations. This helps us uncover hidden performance inefficiencies and establish a baseline for further optimization. Our improved XTR implementation is publicly available in the corresponding GitHub repository<sup>3</sup>.

We present the evaluation of our optimized implementation in Figure 3b. Notably, the optimized implementation’s end-to-end latency is significantly lower than that of the reference implementation ranging from an end-to-end 3.5x speedup on LoTTE pooled to a 6.3x speedup on LoTTE Lifestyle for  $k = 1,000$ . This latency reduction is owed in large parts to a more efficient *scoring* implementation – a 14x speedup on LoTTE Pooled. In particular, this reveals token retrieval as the fundamental bottleneck of the XTR framework. Although the optimized scoring stage accounts for a small fraction of the overall end-to-end latency, it is still slow in absolute terms, ranging from 33ms to 281ms for  $k = 1,000$  on BEIR NFCorpus and LoTTE Pooled, respectively.

## 4 WARP

WARP optimizes retrieval for the refined late interaction architecture introduced in XTR. Seeking to find the best of both the XTR and PLAID worlds, **WARP introduces the novel WARP<sub>SELECT</sub> algorithm for candidate generation, which effectively avoids gathering token-level representations, and proposes an optimized two-stage reduction for faster scoring via a dedicated C++ kernel combined with implicit decompression.** WARP also uses specialized inference runtimes for faster query encoding.

As in XTR, queries and documents are encoded *independently* into embeddings at the token-level using a fine-tuned T5 transformer [16]. To scale to large datasets, document representations are computed in advance and constitute WARP’s index. The similarity between a query  $q$  and document  $d$  is modeled using XTR’s adaptation of ColBERT’s summation of MaxSim operations<sup>4</sup>:

$$S_{d,q} = \sum_{i=1}^n \max_{1 \leq j \leq m} [\hat{A}_{i,j} q_i^T d_j + (1 - \hat{A}_{i,j}) m_i] \quad (1)$$

where  $q$  and  $d$  are the matrix representations of the query and passage embeddings respectively,  $m_i$  denotes the missing similarity estimate for  $q_i$ , and  $\hat{A}$  describes XTR’s alignment matrix. In particular,  $\hat{A}_{i,j} = \mathbb{1}_{[j \in \text{top-}k'_j(\mathbf{d}_{i,j'})]}$  captures whether a document token embedding of a candidate passage was retrieved for a specific query token  $q_i$  as part of the token retrieval stage. We refer to [8, 11] for an intuition behind this choice of scoring function.

### 4.1 Index Construction

Akin to ColBERTv2 [19], WARP’s compression strategy involves applying  $k$ -means clustering to the produced document token embeddings. As in ColBERTv2, we find that using a sample of all passages proportional to the square root of the collection size to generate this clustering performs well in practice. After having clustered the sample of passages, all token-level vectors are encoded and stored as quantized residual vectors to their nearest cluster centroid. Each dimension of the quantized residual vector is a  $b$ -bit encoding of the delta between the centroid and the original uncompressed vector. In particular, these deltas are stored as a sequence of  $128 \cdot \frac{b}{8}$  8-bit values, wherein 128 represents the transformer’s token embedding dimension. Typically, we set  $b = 4$ , i.e., compress each dimension of the residual into a single nibble, for an 8x compression<sup>5</sup>. In this case, each compressed 8-bit value stores 2 indices in the range  $[0, 2^b)$ . Instead of quantizing the residuals uniformly, WARP uses quantiles derived from the empirical distribution to determine bucket boundaries and the corresponding representative values. This process allows WARP to allocate more quantization levels to densely populated regions of the data distribution, thereby minimizing the overall quantization error for residual compression.

### 4.2 Retrieval

Extending PLAID, the retrieval process in the WARP engine is divided into four distinct steps: query encoding, candidate generation, decompression, and scoring. Figure 4 illustrates the retrieval process in WARP. The process starts by encoding the query text into  $q$ , a (query\_maxlen, 128)-dimensional tensor, using the underlying Transformer model.<sup>6</sup> Next, the most similar  $n_{\text{probe}}$  centroids are identified for each of the query\_maxlen query token embeddings.

Subsequently, WARP identifies all document token embeddings belonging to the clusters of the selected centroids and computes their *individual* relevance score. Computing this score involves

<sup>4</sup>Note that we omit the normalization via  $\frac{1}{Z}$ , as we are only interested in the relative ranking between documents and the normalization constant is identical for any retrieved document.

<sup>5</sup>As compared to an uncompressed 32-bit floating point number per dimension.

<sup>6</sup>We set query\_maxlen = 32 in accordance with the XTR paper.

<sup>3</sup><https://github.com/jlscheerer/xtr-eval>